Sri Lanka Institute of Information Technology

SE3090 – Software Engineering Frameworks
Year 3, Semester 1 - 2026

Practical Answer Submission

Practical Sheet No: 04

Student ID	IT24102798
Student Name	Sooriyabandara U.R.G.W.K.
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No.	01.01

Task 01 – Design the Database Schema (3NF)

Table Designs

- **Users Table**

Column	Type	Key / Constraint
users.id	SERIAL / Integer	Primary Key
users.name	TEXT	NOT NULL
users.email	TEXT	NOT NULL, UNIQUE
users.password_hash	TEXT	NOT NULL
users.role	TEXT	NOT NULL, DEFAULT 'Customer'
users.created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

- **MenuItems Table**

Column	Type	Key / Constraint
menu_items.id	SERIAL / Integer	NOT NULL
menu_items.name	TEXT	NOT NULL
menu_items.price	NUMERIC(10,2)	NOT NULL
menu_items.category	TEXT	NOT NULL
menu_items.available	BOOLEAN	NOT NULL, DEFAULT TRUE

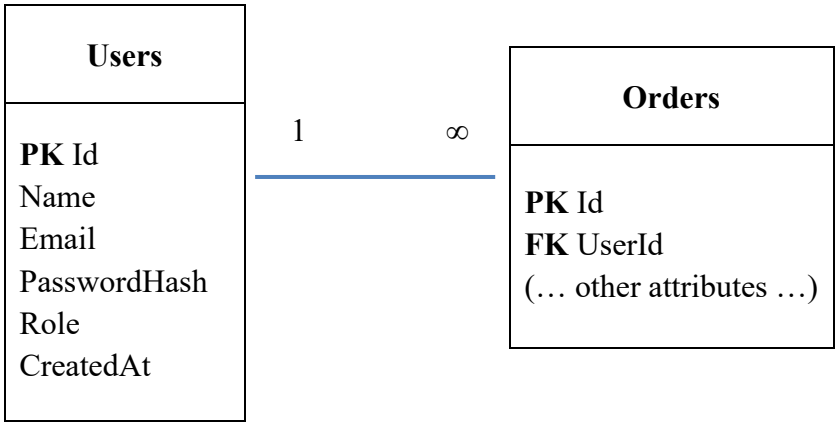
The FK-side answer for the User – Orders relationship

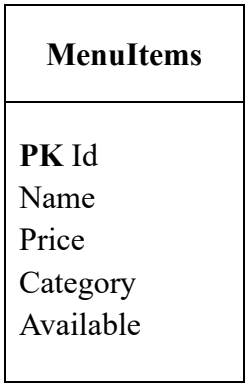
- One “User” can place many Orders, so the relationship is **one-to-many**.
- The foreign key is stored on the many side, which is the “Orders” table. Therefore, “Orders.UserId” will reference “Users.Id”.

The one-line 3NF justification per table

- **Users Table** **:** The Users table is in 3NF because all attributes are atomic and every non-key attribute depends directly on the primary key Id, with no partial or transitive dependencies.
- **MenuItems Table** **:** The MenuItems table is in 3NF because all attributes are atomic and every non-key attribute depends directly on the primary key Id, with no partial or transitive dependencies.

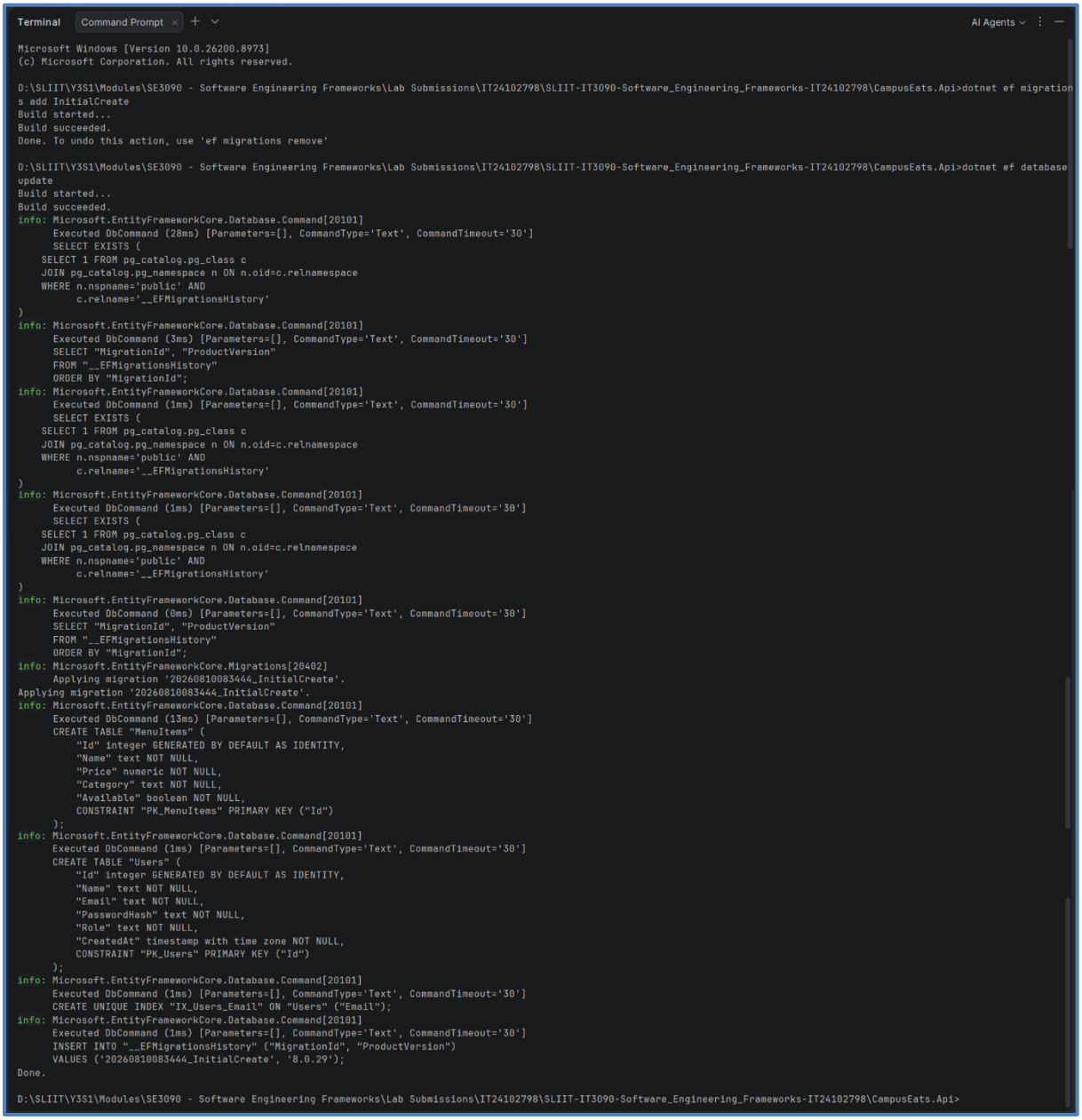
The ER sketch





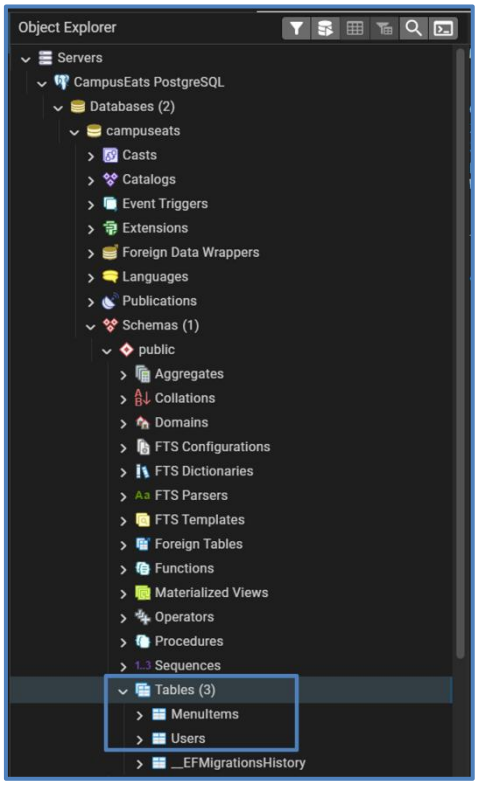
Task 02 – Connect to PostgreSQL with EF Core

Screenshot of Successful Migration



```
Terminal  Command Prompt                                                    AI Agents

Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

D:\SLIIT\Y3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api>dotnet ef migration
s add InitialCreate
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'

D:\SLIIT\Y3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api>dotnet ef database
update
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (28ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
    SELECT 1 FROM pg_catalog.pg_class c
    JOIN pg_catalog.pg_namespace n ON n.oid=c.relnamespace
    WHERE n.nspname='public' AND
          c.relname='__EFMigrationsHistory'
)
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
    SELECT 1 FROM pg_catalog.pg_class c
    JOIN pg_catalog.pg_namespace n ON n.oid=c.relnamespace
    WHERE n.nspname='public' AND
          c.relname='__EFMigrationsHistory'
)
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
    SELECT 1 FROM pg_catalog.pg_class c
    JOIN pg_catalog.pg_namespace n ON n.oid=c.relnamespace
    WHERE n.nspname='public' AND
          c.relname='__EFMigrationsHistory'
)
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Migrations[20402]
      Applying migration '20260810083444_InitialCreate'.
Applying migration '20260810083444_InitialCreate'.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (13ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE "MenuItems" (
          "Id" integer GENERATED BY DEFAULT AS IDENTITY,
          "Name" text NOT NULL,
          "Price" numeric NOT NULL,
          "Category" text NOT NULL,
          "Available" boolean NOT NULL,
          CONSTRAINT "PK_MenuItems" PRIMARY KEY ("Id")
      );
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE "Users" (
          "Id" integer GENERATED BY DEFAULT AS IDENTITY,
          "Name" text NOT NULL,
          "Email" text NOT NULL,
          "PasswordHash" text NOT NULL,
          "Role" text NOT NULL,
          "CreatedAt" timestamp with time zone NOT NULL,
          CONSTRAINT "PK_Users" PRIMARY KEY ("Id")
      );
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE UNIQUE INDEX "IX_Users_Email" ON "Users" ("Email");
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      INSERT INTO "__EFMigrationsHistory" ("MigrationId", "ProductVersion")
      VALUES ('20260810083444_InitialCreate', '8.0.29');
Done.

D:\SLIIT\Y3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api>
```

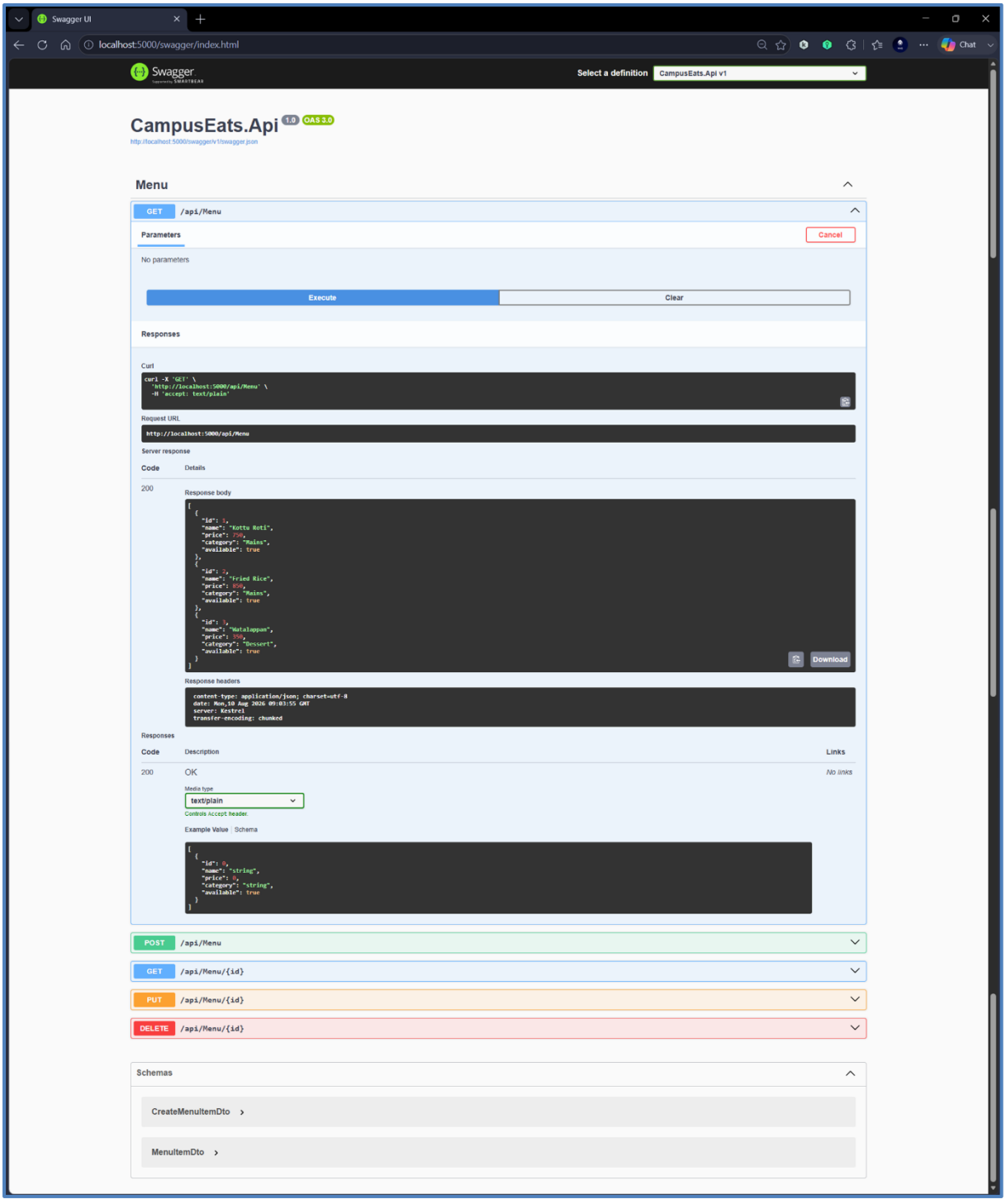
Screenshot of Successful Table Creation of Users and MenuItems from pgAdmin



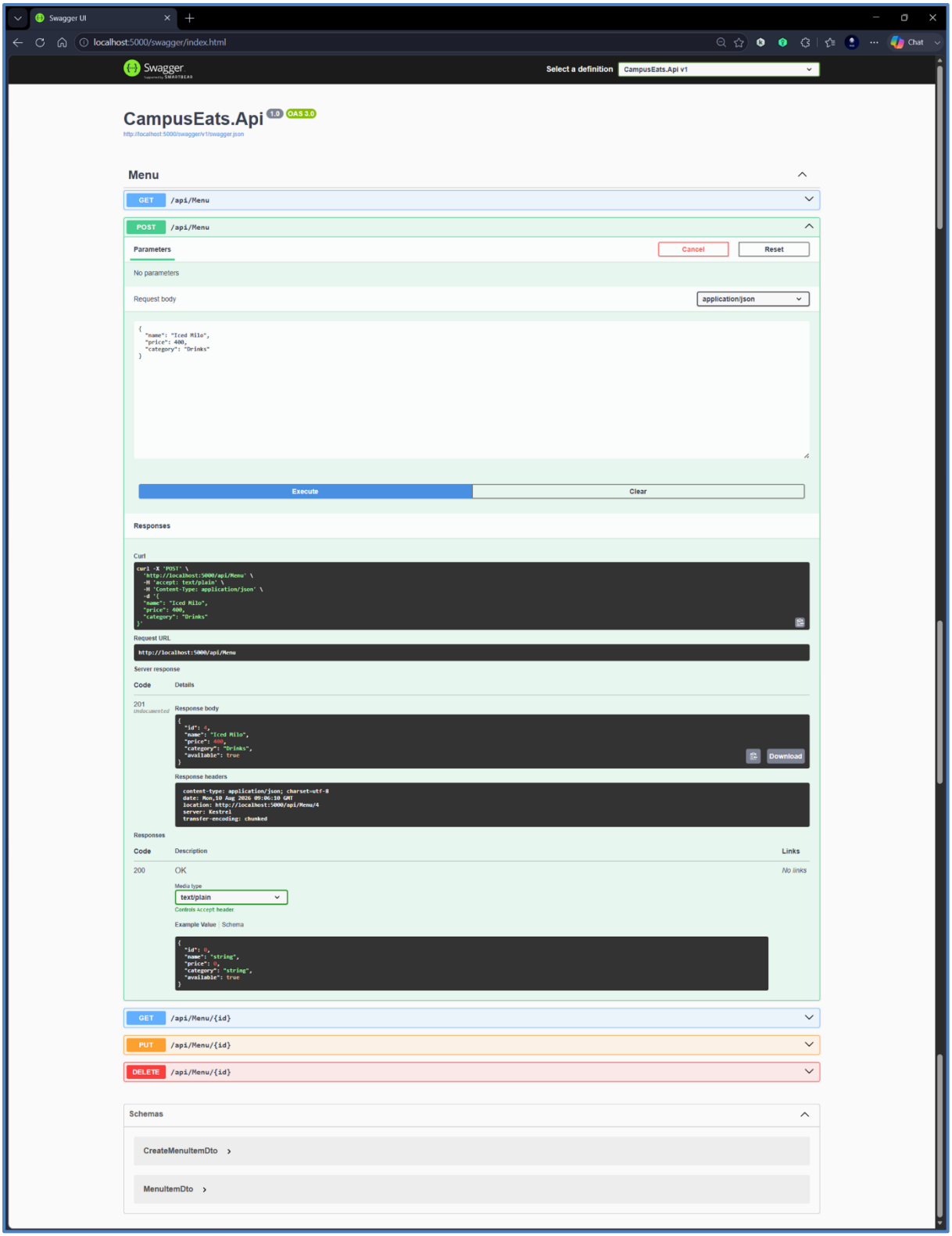
Screenshot of changes made to “MenuService” in “Program.cs”

```
C# Program.cs
// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddScoped<IMenuService, MenuService>();
builder.Services.AddDbContext<AppDbContext>(opt :DbContextOptionsBuilder =>
    opt.UseNpgsql(builder.Configuration // ConfigurationManager
        .GetConnectionString(name: "Default")));

var app :WebApplication = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    if (!db.MenuItems.Any())
    {
        db.MenuItems.AddRange(
            new MenuItem { Name = "Kottu Roti", Price = 750m, Category = "Mains"   },
            new MenuItem { Name = "Fried Rice", Price = 850m, Category = "Mains"   },
            new MenuItem { Name = "Watalappan", Price = 350m, Category = "Dessert" });
        db.SaveChanges();
    }
}

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

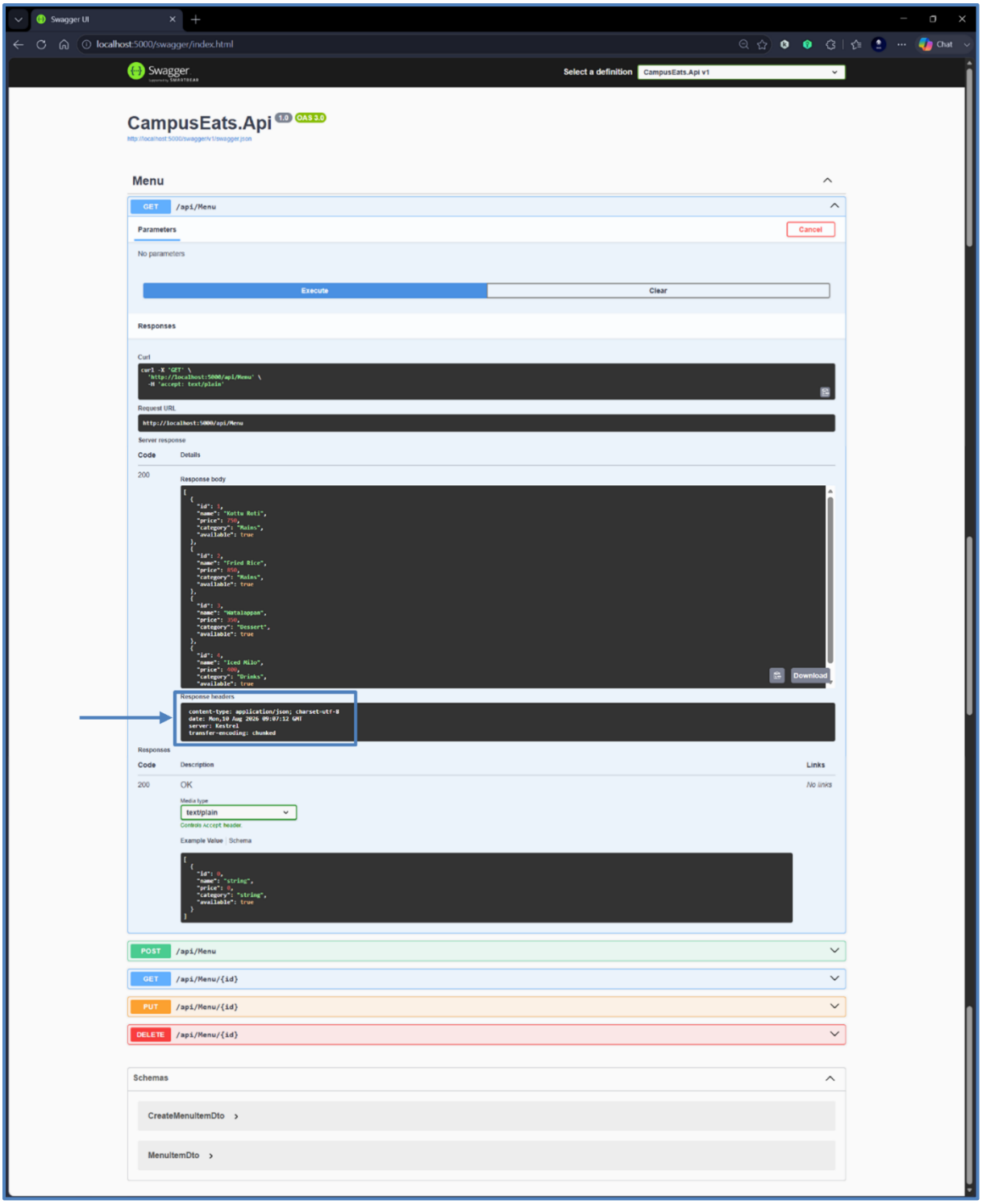
Screenshot of “GET /api/menu” which returns the three seeded dishes from PostgreSQL



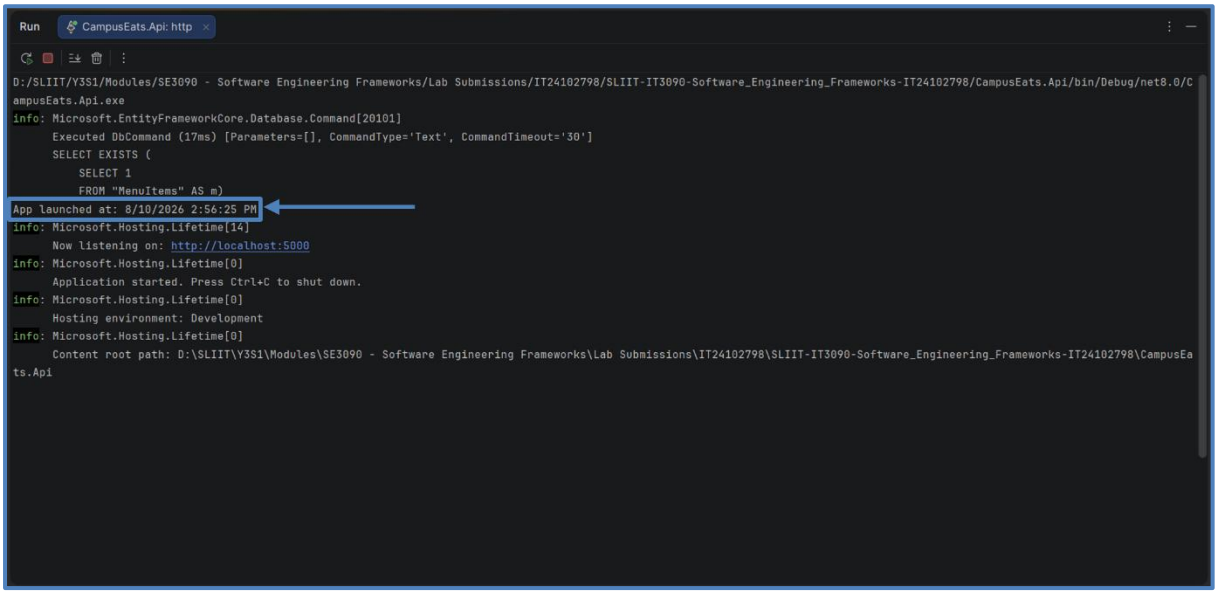
Screenshot of the newly created Dish



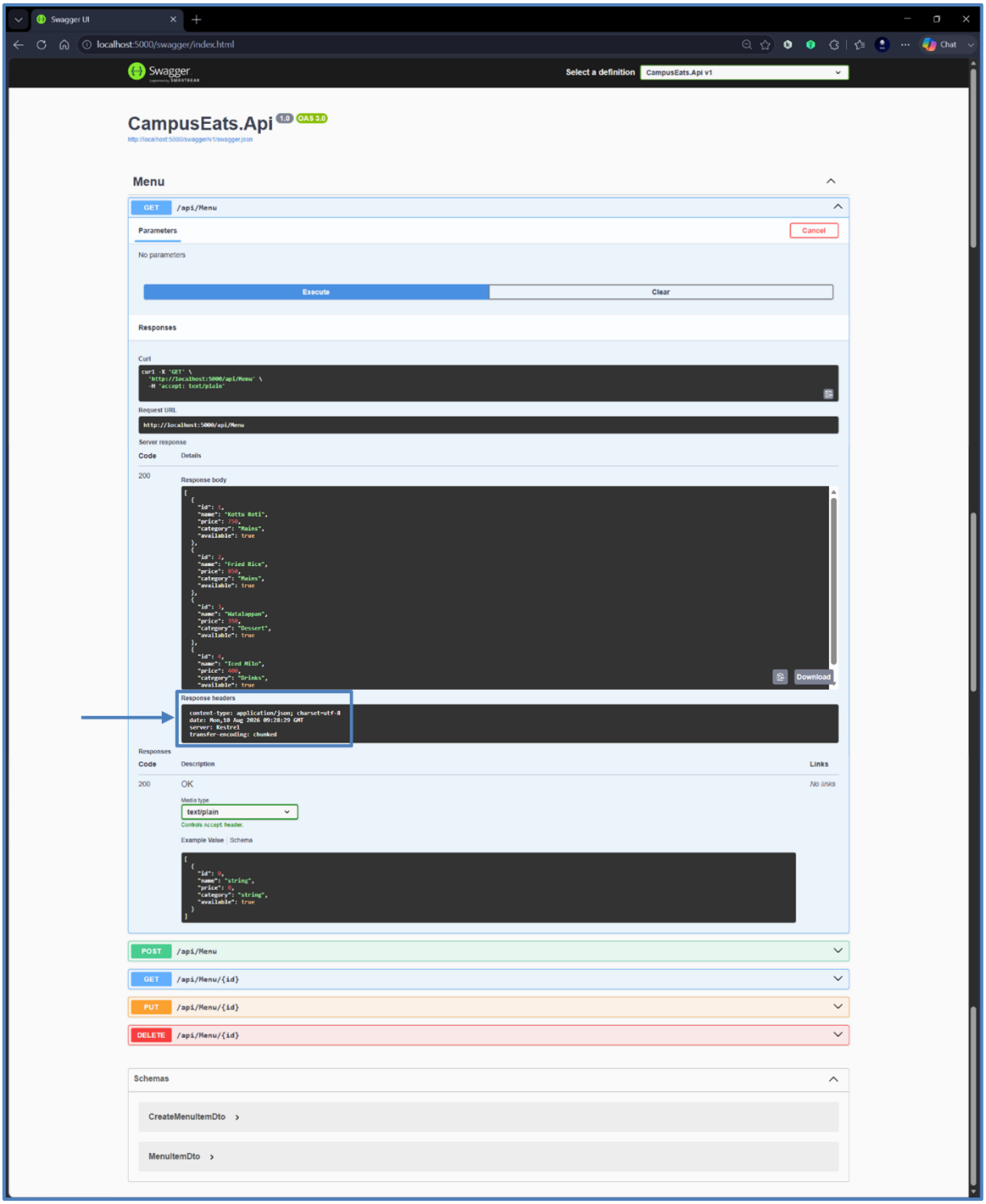
Screenshot of returning all Dishes before the Application Restart
(Time: Mon, 10 Aug 2026 09:07:12 GMT = 8/10/2026 2:37:12 PM)



Screenshot of the Application Restart
(Time: Mon, 10 Aug 2026 09:26:25 GMT = 8/10/2026 2:56:25 PM)

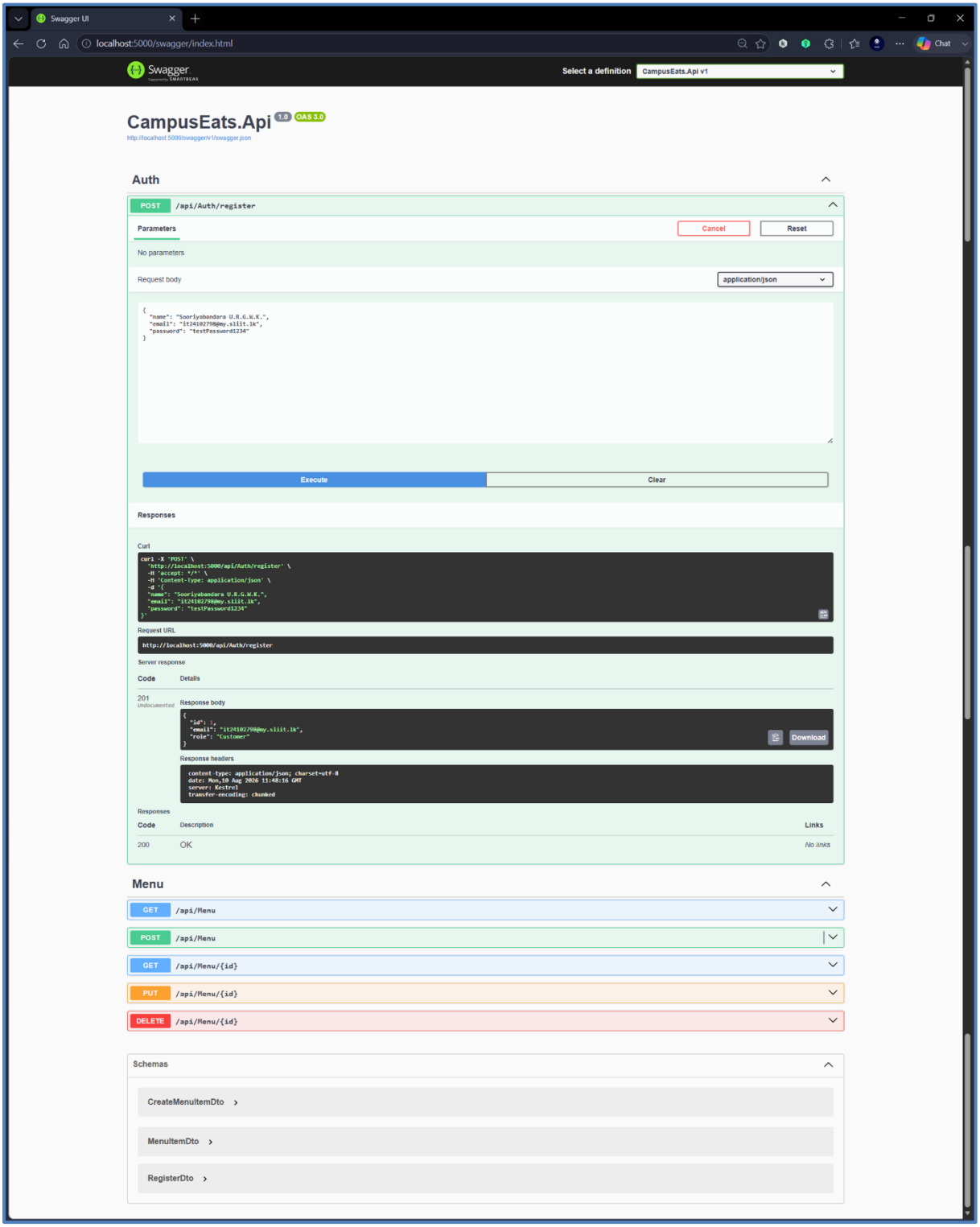


Screenshot of returning all Dishes after the Application Restart
(Time: Mon, 10 Aug 2026 09:28:29 GMT = 8/10/2026 2:58:29 PM)



Task 03 – User Registration with Password Hashing

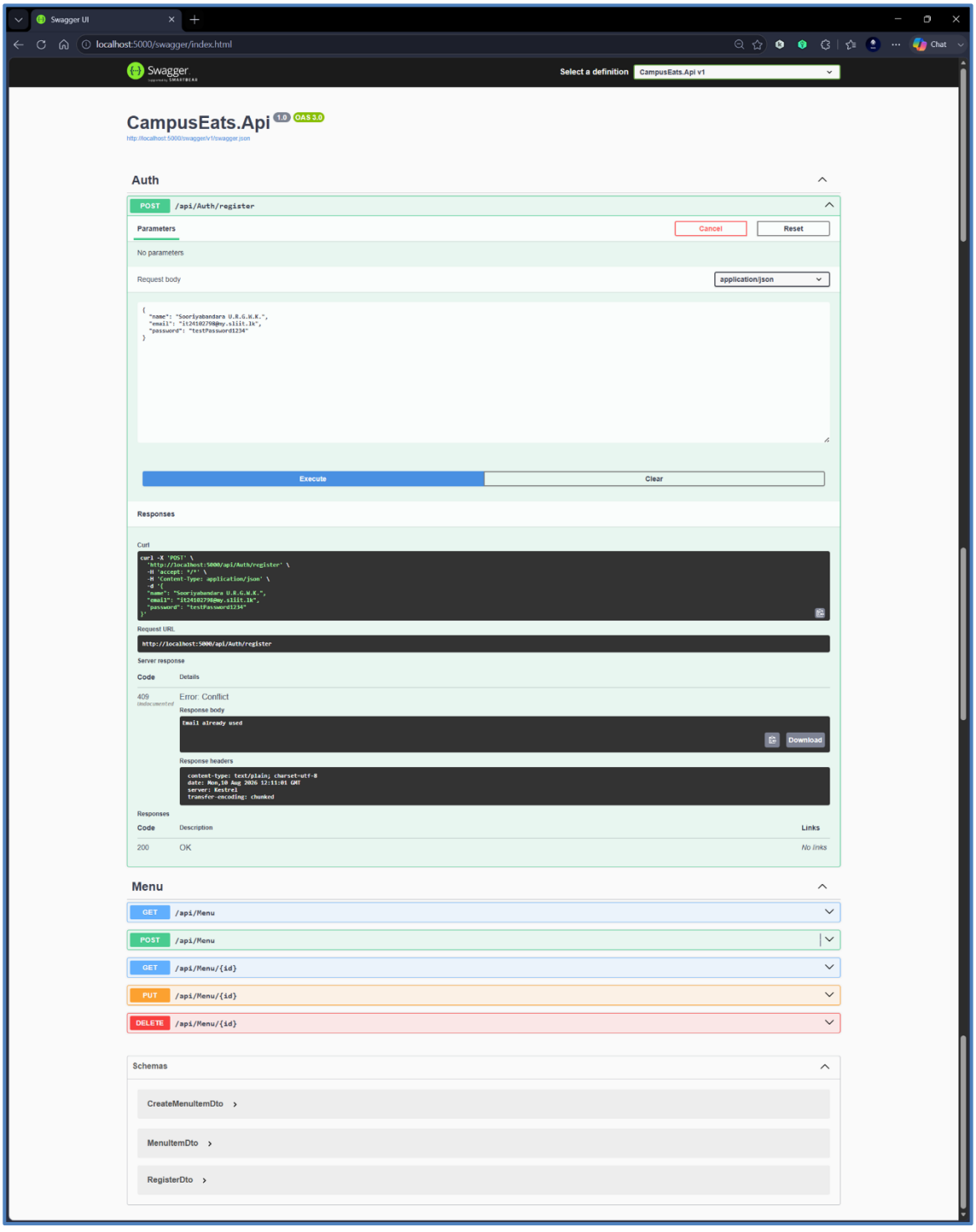
Screenshot of the Working Registration Endpoint that creates a Customer



Screenshot of the Users Table showing the Stored Hash (Never the Plaintext Password)

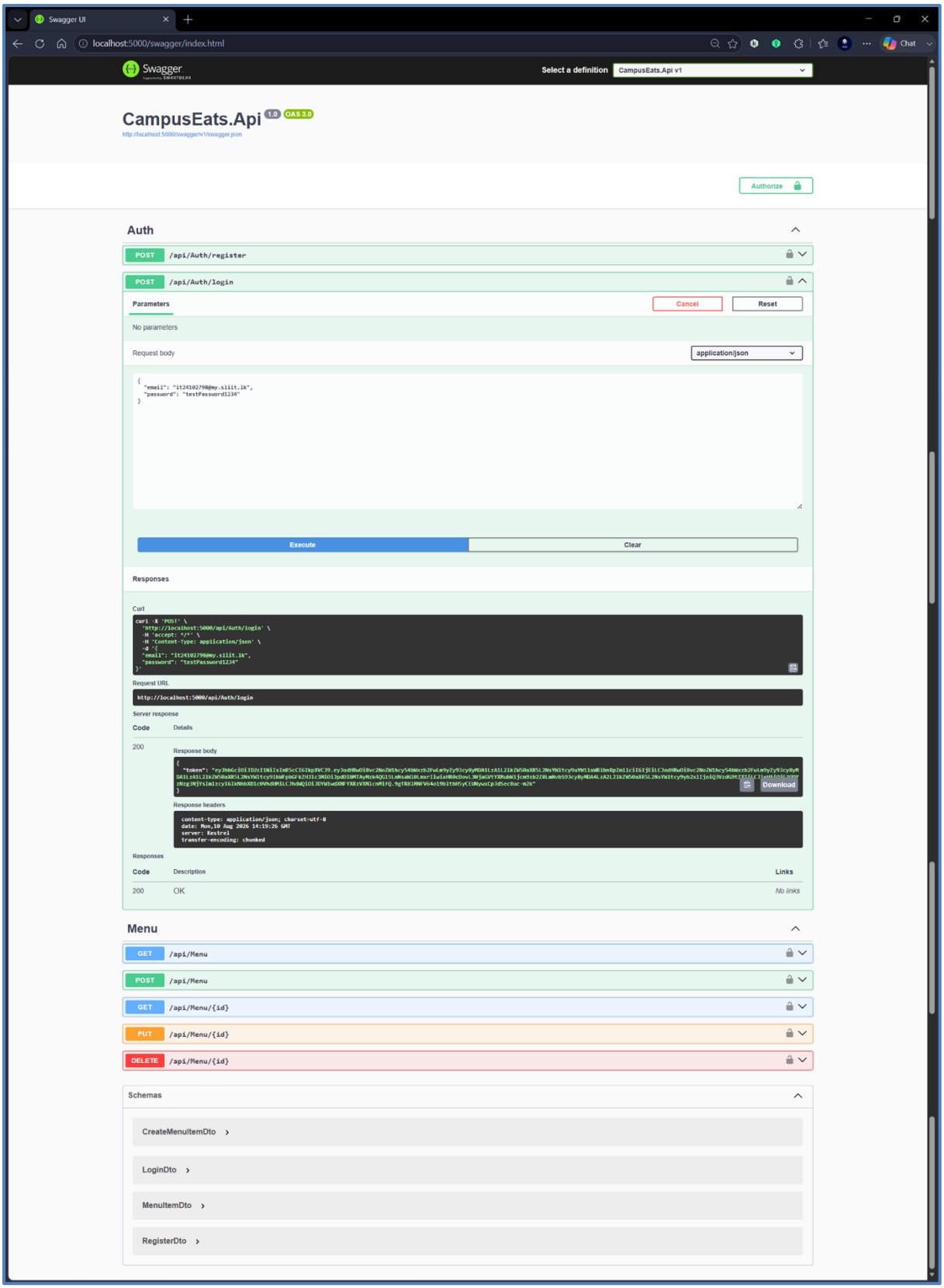
```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\MSI\AppData\Local\Programs\pgAdmin 4\runtime>"C:\Users\MSI\AppData\Local\Programs\pgAdmin 4\runtime\psql.exe" "host=localhost port=5432 dbname=campuseats user=postgres
sslmode=prefer connect_timeout=10 keepalives=1 keepalives_idle=30 keepalives_interval=10 keepalives_count=3" 2>>&1
psql (18.4)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

campuseats=# SELECT * FROM "Users";
 Id |          Name           |         Email          |                        PasswordHash                        |   Role   |          CreatedAt
----+-------------------------+------------------------+------------------------------------------------------------+----------+----------------------------
  1 | Sooriyabandara U.R.G.W.K. | it24102798@my.sliit.lk | $2a$11$3WYRN14E.L1V0stYQlexRuK1a9ggAqXO.MhtMg4.v32.VA3gfRaxW | Customer | 2026-08-10 11:48:15.894732+00
(1 row)


campuseats=#
```

Screenshot of the *409 response* when the same email is reused

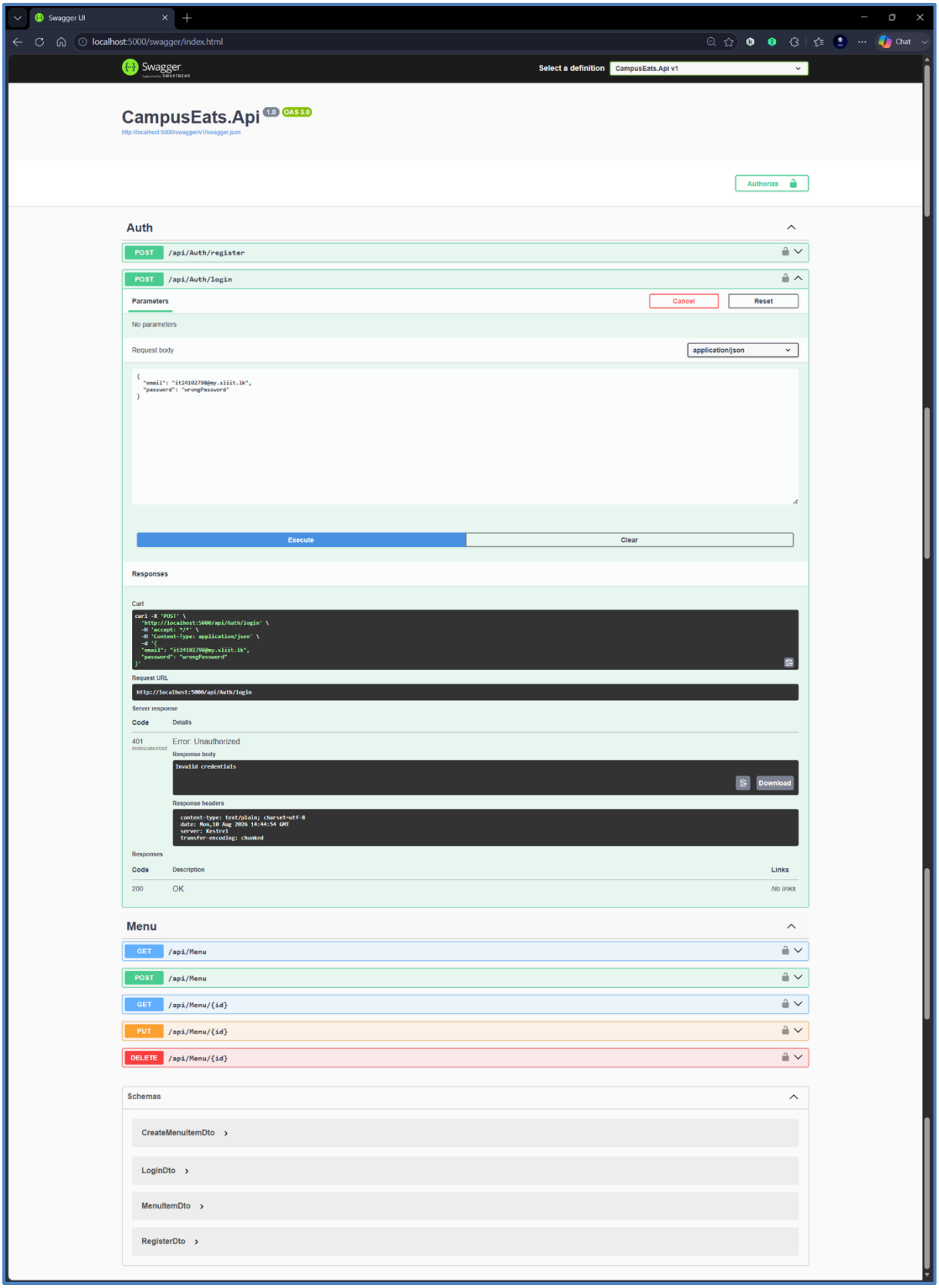


Task 04 – Login & JWT Token Generation

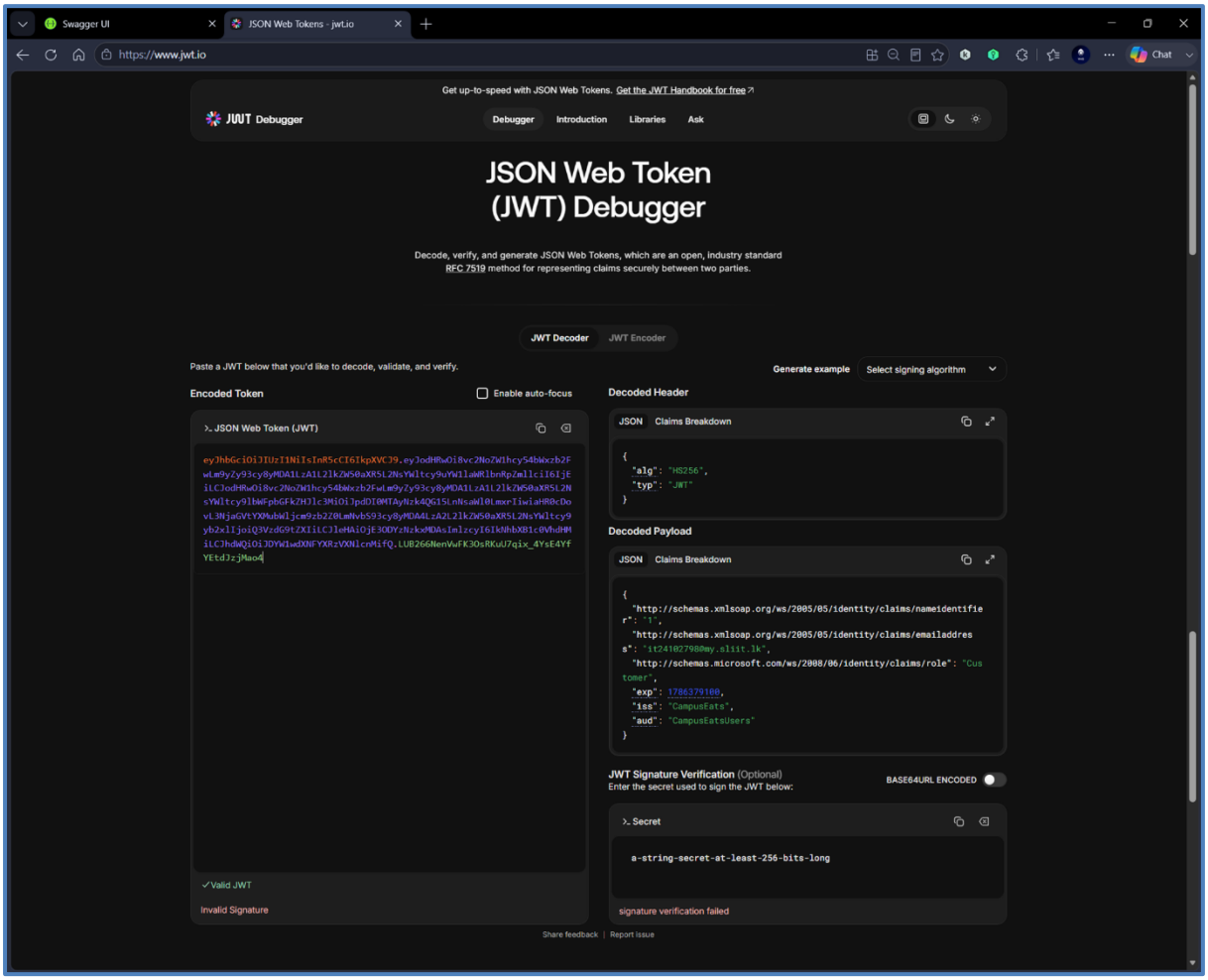
Screenshot of Login API Endpoint returning a JWT for Valid Credentials



Screenshot of Login API Endpoint returning *401 Unauthorized* for Invalid Credentials



Screenshot of the Token decoded at *jwt.io* showing the Role Claim

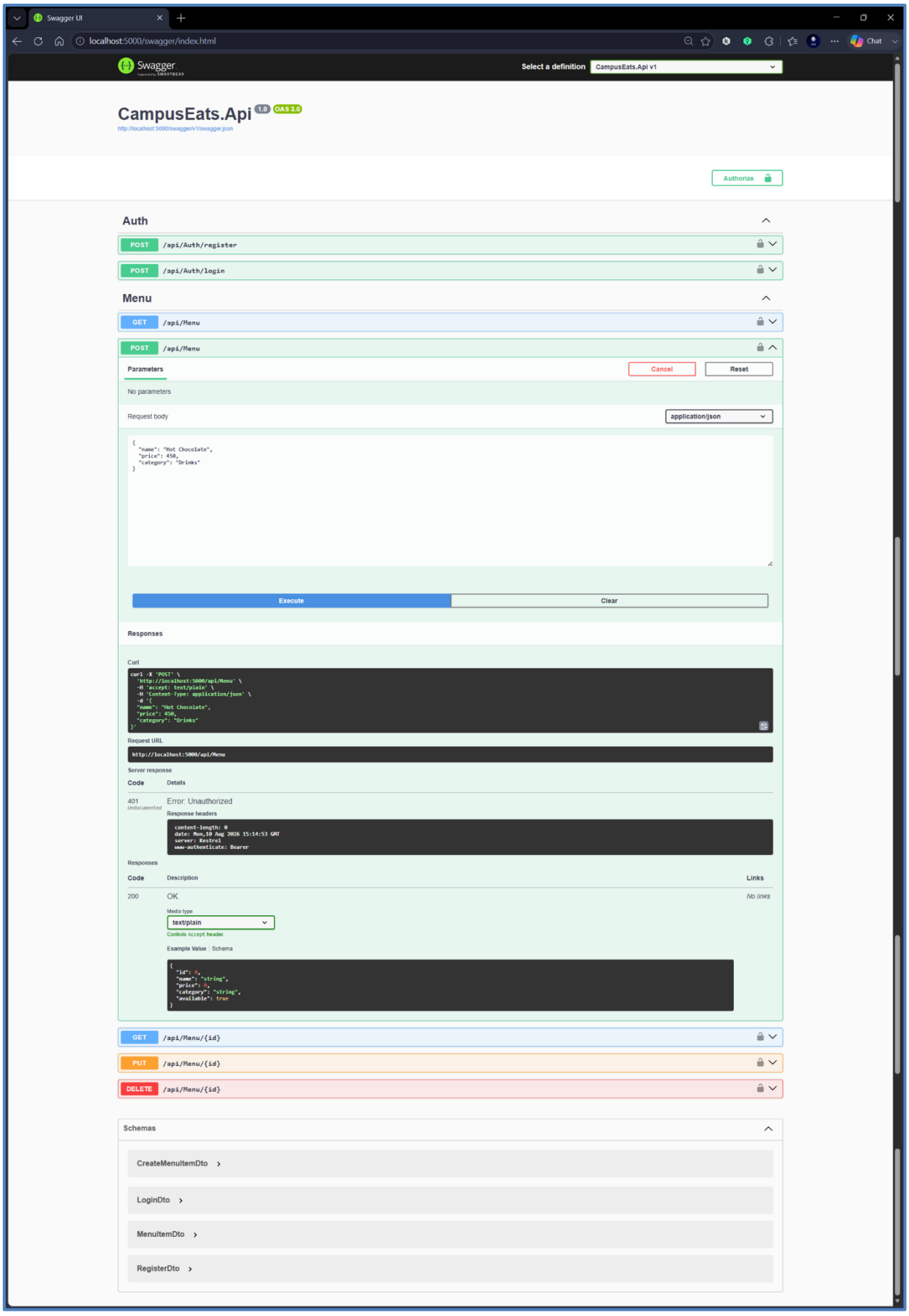


One-Line Note on what the Signature is for,

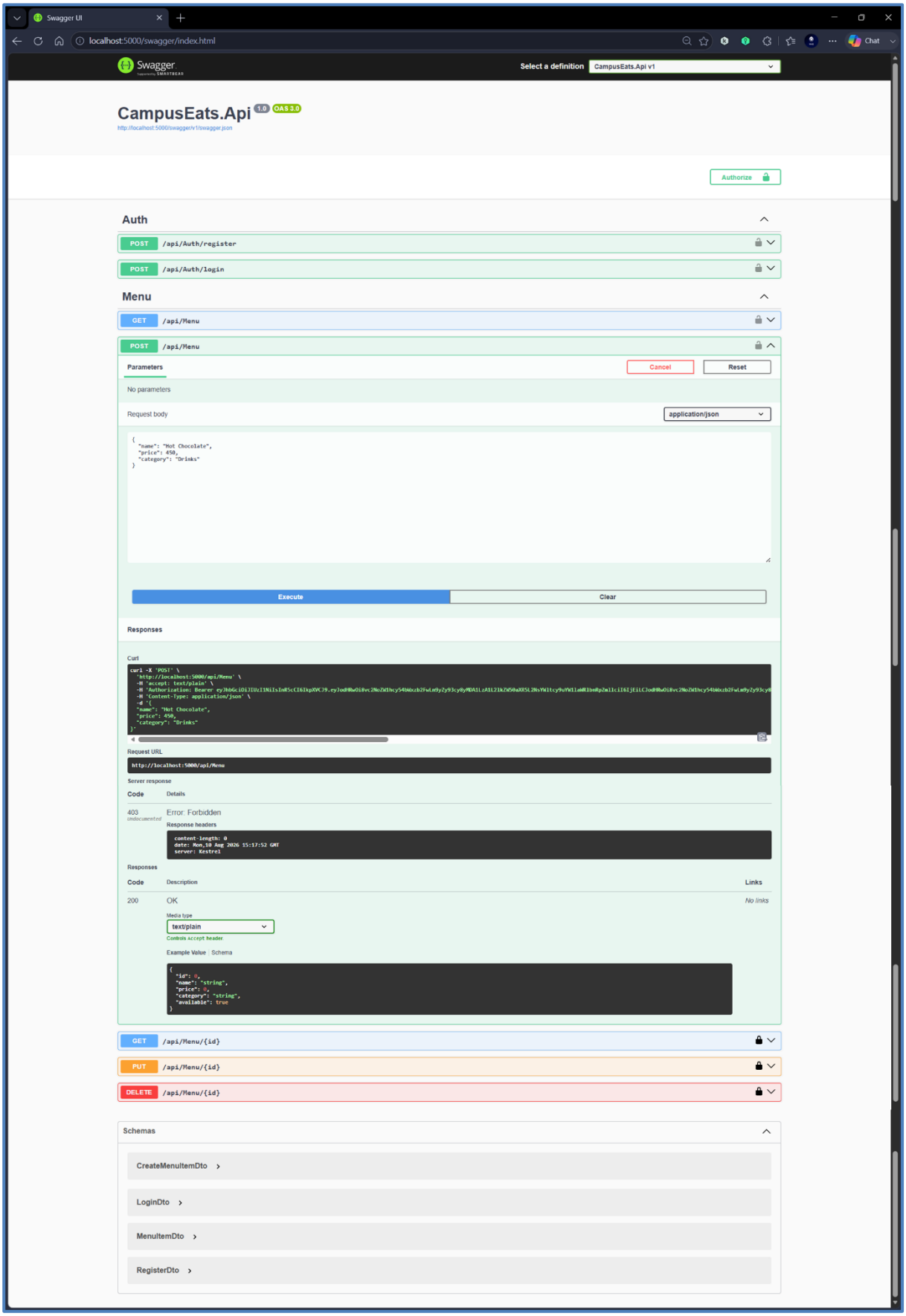
- The JWT signature verifies that the token was issued by the server and that its contents have not been modified or forged.

Task 05 – Protect Endpoints with RBAC & Test the Secure Flow

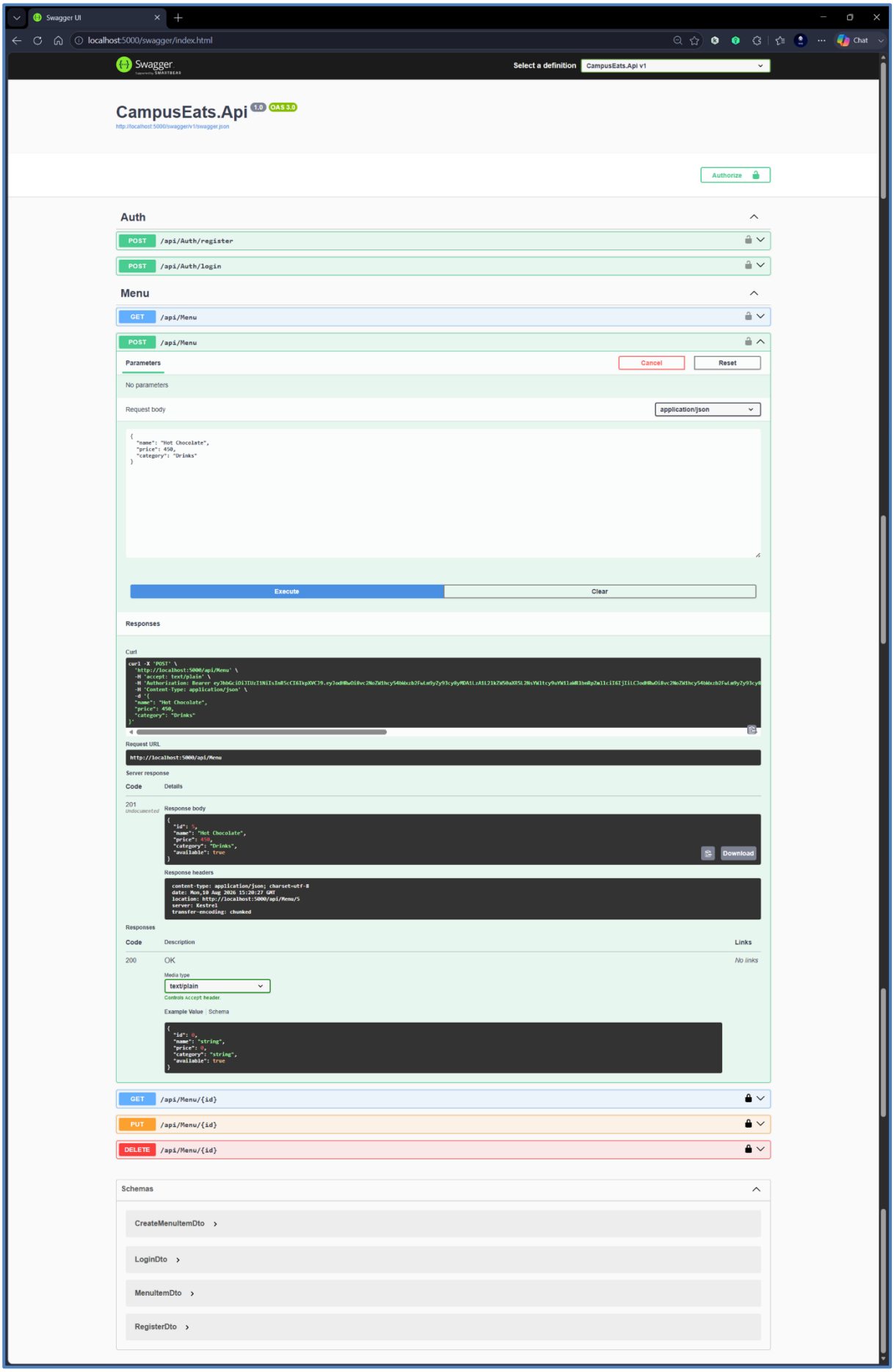
Screenshot of POST /api/Menu with *401 Unauthorized* for accessing without a Token



Screenshot of POST /api/Menu with *403 Forbidden* for accessing with Customer JWT Token



Screenshot of POST /api/Menu with *201 Created* for accessing with Admin JWT Token



Short Written Justifications

- **Why store a password hash instead of the password?**
  - A password hash is stored instead of the plaintext password because a one-way hash prevents the original password from being directly recovered from the database. BCrypt also uses salting, making password hashes harder to crack even if the database is compromised.

- **Why use a stateless JWT instead of a server-side session?**
  - A stateless JWT allows the server to authenticate requests using the information contained in the token without maintaining a session record for every logged-in user. This makes the API easier to scale because requests can be handled independently by different server instances as long as they can validate the token.

- **Why use role-based access control instead of checking individual users in every method?**
  - Role-based access control groups users according to their responsibilities, such as Customer and Admin, instead of hard-coding individual user checks into every endpoint. This makes authorization easier to maintain and allows the same access rules to be applied consistently to many users and endpoints.

Quick Test – Knowledge Check

Multiple Choice (Choose One)

Q1. (c) primary key

Q2. (b) the Orders (many) side

Q3. (c) 403 Forbidden

Q4. (b) as a salted one-way hash

Q5. (c) signature

Database Design

Q6.

- It breaks First Normal Form (1NF) because the course columns represent repeating groups instead of storing one atomic value per field. It is better to be redesigned using separate Students, Courses, and StudentCourses tables, where StudentCourses contain the foreign keys StudentId and CourseId to represent the many-to-many relationship.

Authentication

Q7.

- **Step 01:** The user submits their email and password to the login endpoint, and the server verifies the password against the stored BCrypt hash.
- **Step 02:** If the credentials are valid, the server creates and signs a JWT containing claims such as the user's ID, email, and role.
- **Step 03:** The server returns the JWT to the client, which stores it and sends it with future authenticated requests as a Bearer token.

Authorization Scenario

Q8.

- It is 403 Forbidden because the Customer is already authenticated with a valid JWT, but their Customer role is not permitted to delete menu items. The endpoint is protected by [Authorize(Roles = "Admin")], which allows only users with the Admin role to perform the operation.